

Practical Implementation Sheet

Department: CSE-AI

Semester: 5

Academic Year: 2026-27

Division: C

Batch: 3

Practical No: 2

Roll no: 68

PRN No: 12415154

Name of Student: Kalpesh Patil

Subject Name: Deep Learning

Problem Statement:

Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm:

Forward Propagation and Backpropagation using TensorFlow/Keras

- 1. Import the required libraries.**
- 2. Load the MNIST (or Fashion-MNIST) dataset.**
- 3. Preprocess the dataset by normalizing pixel values and reshaping the input images.**
- 4. Split the dataset into training and testing sets.**
- 5. Accept the following inputs from the user:**
 - **Number of learning rates**
 - **Learning rate values**
 - **Number of epoch values**
 - **Epoch values**
- 6. Create a Multilayer Perceptron (MLP) model by specifying:**
 - **Input layer**
 - **Hidden layers with ReLU activation**
 - **Output layer with Softmax activation**
 - **Adam optimizer**

- **Learning rate**

7. Perform forward propagation by passing the input data through the network to generate predictions.

8. Perform backpropagation by computing the loss, calculating gradients, and updating the network weights using the Adam optimizer.

9. Train the model for each specified learning rate and epoch combination.

10. Evaluate the trained model on the test dataset by calculating:

- **Test Accuracy**
- **Test Loss**

11. Generate visualizations to analyze model performance, such as:

- **Accuracy vs Learning Rate**
- **Loss vs Learning Rate**
- **Accuracy vs Epochs**
- **Loss vs Epochs**
- **Training vs Validation Accuracy**
- **Training vs Validation Loss**
- **Comparison Bar Charts**
- **Accuracy Heatmap**
- **Confusion Matrix**
- **Sample Predictions**

12. Display the comparison table containing the learning rate, number of epochs, loss, and accuracy for all experiments.

13. Identify the best-performing hyperparameter combination based on the highest accuracy and lowest loss.

14. Analyze the effect of different learning rates and epochs on the convergence, training stability, and overall model performance.

```

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
import pandas as pd

# MNIST
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()

x_train = x_train.reshape(-1, 784) / 255.0
x_test = x_test.reshape(-1, 784) / 255.0

# -----
# Model
# -----

def create_model(lr):

    model = Sequential([
        Dense(128, activation='relu', input_shape=(784,)),
        Dense(64, activation='relu'),
        Dense(10, activation='softmax')
    ])

```

Code:

```

    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model

# -----
# USER INPUT
# -----

num_lr = int(input("Enter number of learning rates: "))

learning_rates = []

for i in range(num_lr):
    lr = float(input(f"Enter Learning Rate {i+1}: "))
    learning_rates.append(lr)

print()

num_epochs = int(input("Enter number of epoch values: "))

epochs_list = []

for i in range(num_epochs):
    ep = int(input(f"Enter Epoch Value {i+1}: "))
    epochs_list.append(ep)

```

```

print("\nTraining Started...\n")

# -----
# Effect of Learning Rate
# -----

lr_results = []

for lr in learning_rates:

    model = create_model(lr)

    model.fit(
        x_train,
        y_train,
        epochs=5,
        batch_size=128,
        verbose=0
    )

    loss, acc = model.evaluate(x_test, y_test, verbose=0)

    lr_results.append([lr, 5, loss, acc])

lr_df = pd.DataFrame(
    lr_results,
    columns=["Learning Rate", "Epochs", "Loss", "Accuracy"]
)

```

```

print("Learning Rate Results")
print(lr_df)
print()

# -----
# Effect of Epochs
# -----

epoch_results = []

for ep in epochs_list:

    model = create_model(0.001)

    model.fit(
        x_train,
        y_train,
        validation_split=0.2,
        epochs=ep,
        batch_size=128,
        verbose=0
    )

    loss, acc = model.evaluate(x_test, y_test, verbose=0)

    epoch_results.append([0.001, ep, loss, acc])

```

```

epoch_df = pd.DataFrame(
    epoch_results,
    columns=["Learning Rate", "Epochs", "Loss", "Accuracy"]
)

print("Epoch Results")
print(epoch_df)
print()

# -----
# Final Comparison Table
# -----

comparison_df = pd.concat([lr_df, epoch_df], ignore_index=True)

comparison_df["Accuracy"] = comparison_df["Accuracy"].round(4)
comparison_df["Loss"] = comparison_df["Loss"].round(4)

print("=" * 60)
print("FINAL COMPARISON TABLE")
print("=" * 60)
print(comparison_df)

print("\nBest Configuration\n")
best = comparison_df.loc[comparison_df["Accuracy"].idxmax()]
print(best)

```

```
plt.figure(figsize=(6,4))

plt.plot(
    lr_df["Learning Rate"],
    lr_df["Accuracy"],
    marker='o'
)

plt.xscale("log")

plt.xlabel("Learning Rate")
plt.ylabel("Accuracy")
plt.title("Learning Rate vs Accuracy")
plt.grid(True)

plt.show()
```

```
# -----
# Plot Epoch vs Accuracy
# -----
```

```
plt.figure(figsize=(6,4))

plt.plot(
    epoch_df["Epochs"],
    epoch_df["Accuracy"],
    marker='o'
)

plt.xlabel("Epochs")
```

```
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Epochs vs Accuracy")
plt.grid(True)

plt.show()
```

```
# -----
# Train Final Model
# -----
```

```
best_epoch = max(epochs_list)

model = create_model(0.001)

history = model.fit(
    x_train,
    y_train,
    validation_split=0.2,
    epochs=best_epoch,
    batch_size=128,
    verbose=1
)
```

```

plt.figure(figsize=(6,4))

plt.plot(history.history["accuracy"])
plt.plot(history.history["val_accuracy"])

plt.legend(["Train", "Validation"])
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training Accuracy")

plt.show()

# -----
# Loss Graph
# -----

plt.figure(figsize=(6,4))

plt.plot(history.history["loss"])
plt.plot(history.history["val_loss"])

plt.legend(["Train", "Validation"])
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Loss")

plt.show()

plt.figure(figsize=(7,5))
plt.plot(lr_df["Learning Rate"], lr_df["Loss"], marker='o')

plt.figure(figsize=(7,5))
plt.plot(lr_df["Learning Rate"], lr_df["Loss"], marker='o')
plt.xscale("log")
plt.xlabel("Learning Rate")
plt.ylabel("Loss")
plt.title("Learning Rate vs Loss")
plt.grid(True)
plt.show()

```

```

plt.figure(figsize=(10,5))

labels = []

for i in range(len(comparison_df)):
    labels.append(f"LR={comparison_df.iloc[i]['Learning Rate']}\nE={comparison_df.iloc[i]['Epochs']}")

plt.bar(labels, comparison_df["Accuracy"])

plt.ylabel("Accuracy")
plt.title("Accuracy Comparison")
plt.xticks(rotation=45)
plt.show()

import seaborn as sns
heatmap_data = comparison_df.pivot_table(
    values="Accuracy",
    index="Epochs",
    columns="Learning Rate"
)

plt.figure(figsize=(8,6))

sns.heatmap(
    heatmap_data,
    annot=True,
    cmap="viridis",
    fmt=".4f"
)

plt.title("Accuracy Heatmap")
plt.show()

```

```

# -----
# Final Test Accuracy
# -----

loss, acc = model.evaluate(x_test, y_test)

print("\nFinal Test Accuracy :", round(acc, 4))
print("Final Test Loss      :", round(loss, 4))

```

Output:

Enter number of learning rates: 5
Enter Learning Rate 1: 1
Enter Learning Rate 2: 0.01
Enter Learning Rate 3: 0.001
Enter Learning Rate 4: 0.0001
Enter Learning Rate 5: 0.00001

Enter number of epoch values: 3
Enter Epoch Value 1: 5
Enter Epoch Value 2: 10
Enter Epoch Value 3: 20

Training Started...

Learning Rate Results

	Learning Rate	Epochs	Loss	Accuracy
0	1.00000	5	2.322692	0.1000
1	0.01000	5	0.382695	0.8661
2	0.00100	5	0.360183	0.8692
3	0.00010	5	0.428152	0.8476
4	0.00001	5	0.768949	0.7496

Epoch Results

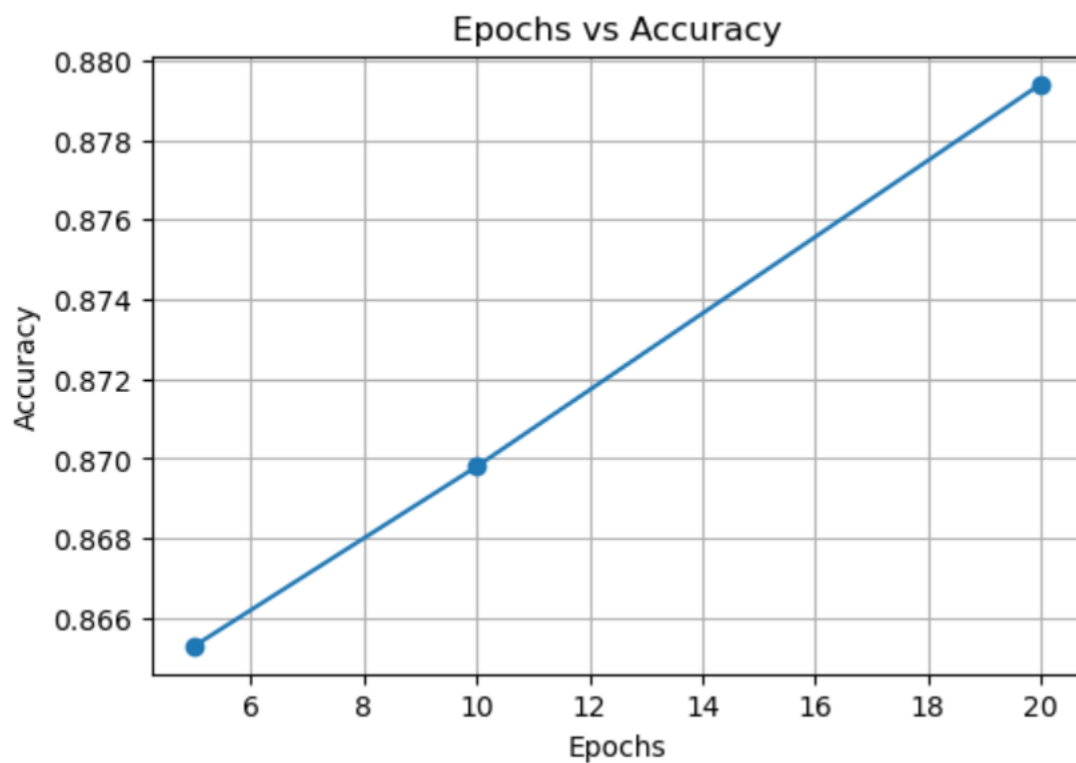
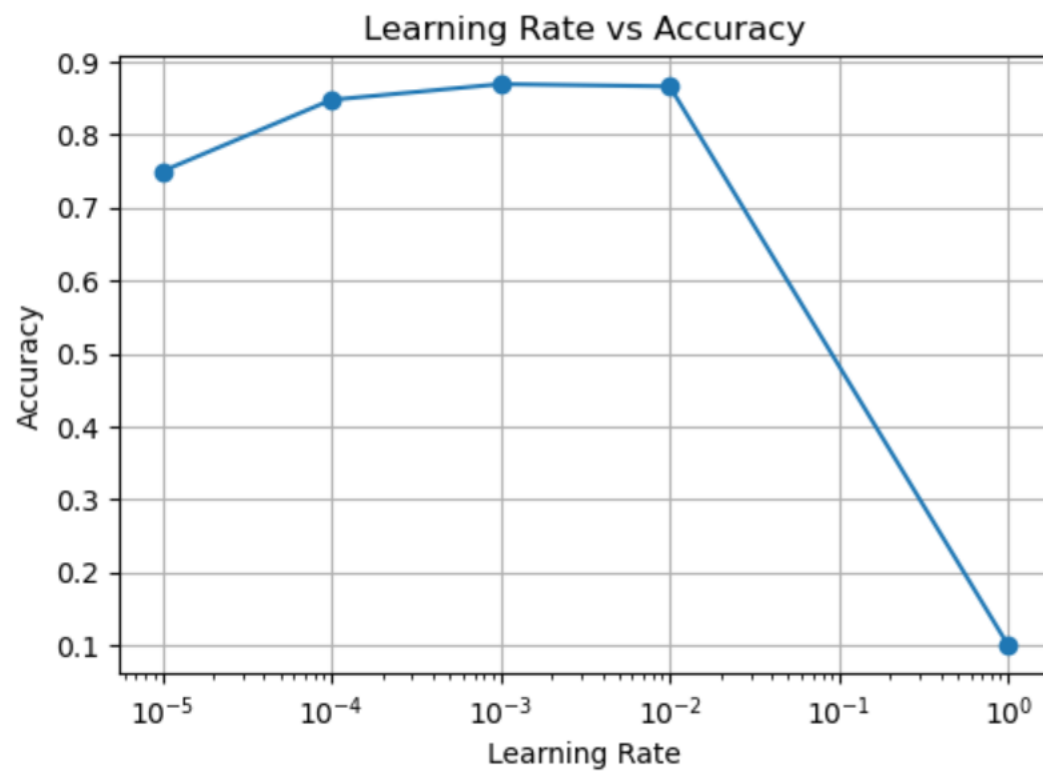
	Learning Rate	Epochs	Loss	Accuracy
0	0.001	5	0.371754	0.8653
1	0.001	10	0.376071	0.8698
2	0.001	20	0.369676	0.8794

FINAL COMPARISON TABLE

	Learning Rate	Epochs	Loss	Accuracy
0	1.00000	5	2.3227	0.1000
1	0.01000	5	0.3827	0.8661
2	0.00100	5	0.3602	0.8692
3	0.00010	5	0.4282	0.8476
4	0.00001	5	0.7689	0.7496
5	0.00100	5	0.3718	0.8653
6	0.00100	10	0.3761	0.8698
7	0.00100	20	0.3697	0.8794

Best Configuration

Learning Rate 0.0010
Epochs 20.0000
Loss 0.3697
Accuracy 0.8794
Name: 7, dtype: float64



Epoch 1/20
 375/375 ————— 6s 10ms/step - accuracy: 0.8012 - loss: 0.5798 - val_accuracy: 0.8391 - val_loss: 0.4566

Epoch 2/20
 375/375 ————— 3s 8ms/step - accuracy: 0.8545 - loss: 0.4130 - val_accuracy: 0.8578 - val_loss: 0.3989

Epoch 3/20
 375/375 ————— 3s 8ms/step - accuracy: 0.8665 - loss: 0.3712 - val_accuracy: 0.8693 - val_loss: 0.3719

Epoch 4/20
 375/375 ————— 3s 9ms/step - accuracy: 0.8760 - loss: 0.3411 - val_accuracy: 0.8684 - val_loss: 0.3636

Epoch 5/20
 375/375 ————— 3s 8ms/step - accuracy: 0.8831 - loss: 0.3190 - val_accuracy: 0.8765 - val_loss: 0.3378

Epoch 6/20
 375/375 ————— 3s 9ms/step - accuracy: 0.8902 - loss: 0.3022 - val_accuracy: 0.8818 - val_loss: 0.3245

Epoch 7/20
 375/375 ————— 3s 8ms/step - accuracy: 0.8949 - loss: 0.2858 - val_accuracy: 0.8803 - val_loss: 0.3333

Epoch 8/20
 375/375 ————— 3s 8ms/step - accuracy: 0.8988 - loss: 0.2755 - val_accuracy: 0.8814 - val_loss: 0.3306

Epoch 9/20
 375/375 ————— 4s 10ms/step - accuracy: 0.9030 - loss: 0.2655 - val_accuracy: 0.8860 - val_loss: 0.3149

Epoch 10/20
 375/375 ————— 5s 10ms/step - accuracy: 0.9050 - loss: 0.2568 - val_accuracy: 0.8739 - val_loss: 0.3441

Epoch 11/20
 375/375 ————— 4s 12ms/step - accuracy: 0.9082 - loss: 0.2478 - val_accuracy: 0.8843 - val_loss: 0.3225

Epoch 12/20
 375/375 ————— 4s 11ms/step - accuracy: 0.9126 - loss: 0.2365 - val_accuracy: 0.8865 - val_loss: 0.3257

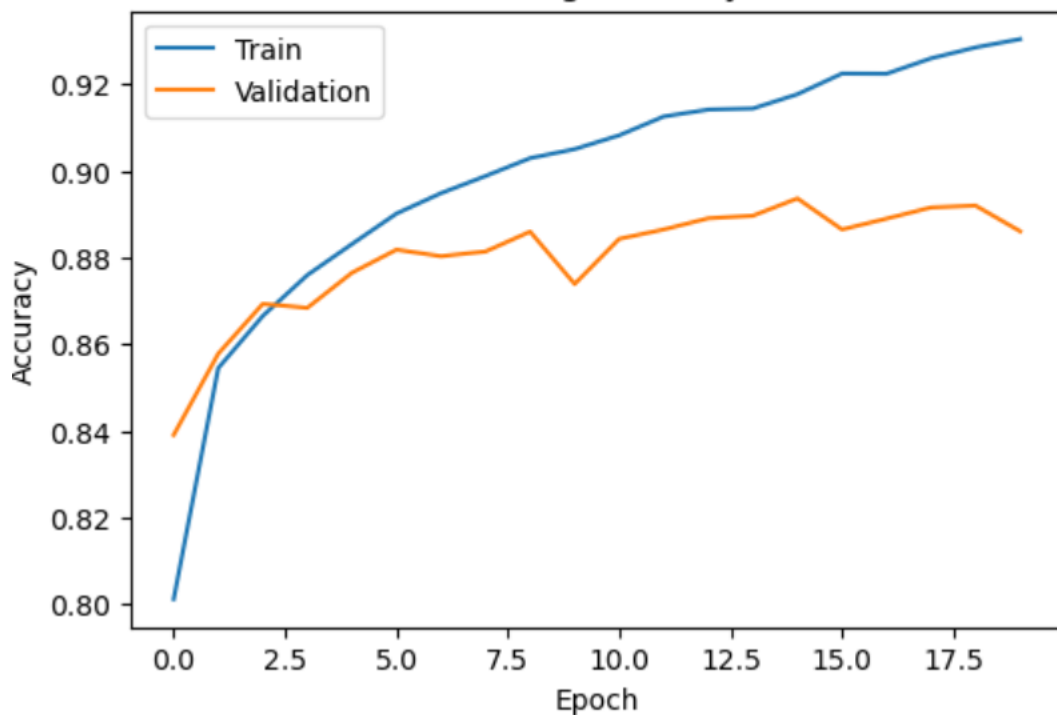
Epoch 13/20
 375/375 ————— 4s 12ms/step - accuracy: 0.9142 - loss: 0.2305 - val_accuracy: 0.8891 - val_loss: 0.3144

Epoch 14/20
 375/375 ————— 5s 11ms/step - accuracy: 0.9144 - loss: 0.2250 - val_accuracy: 0.8897 - val_loss: 0.3131

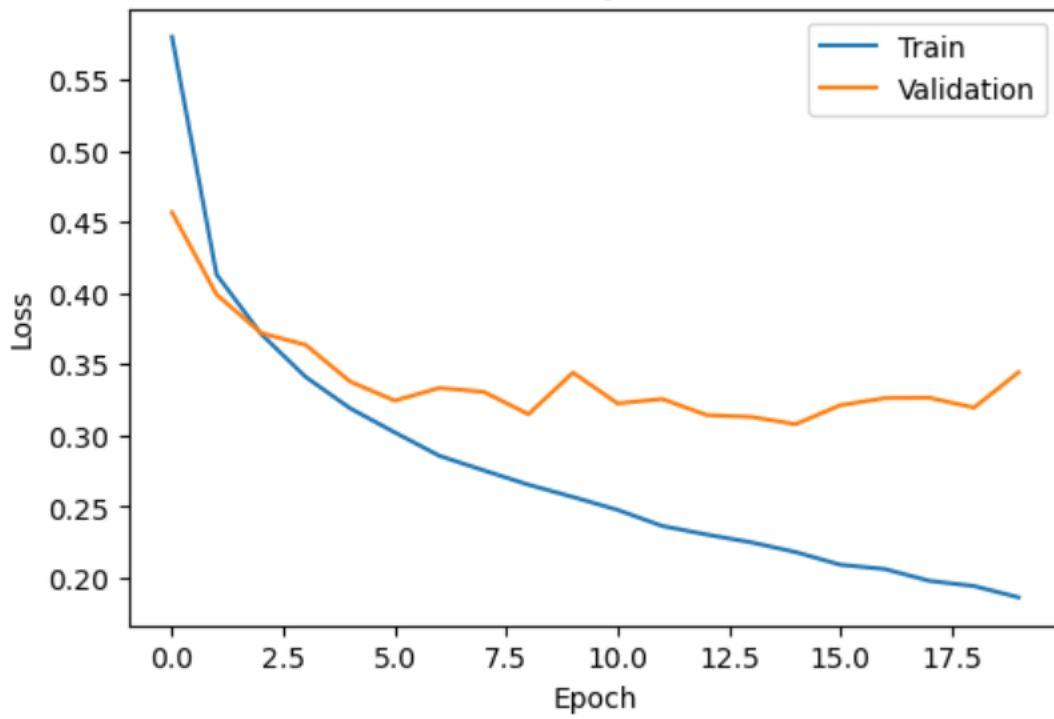
Epoch 15/20
 375/375 ————— 5s 11ms/step - accuracy: 0.9177 - loss: 0.2182 - val_accuracy: 0.8937 - val_loss: 0.3079

Epoch 16/20
 375/375 ————— 4s 11ms/step - accuracy: 0.9225 - loss: 0.2093 - val_accuracy: 0.8865 - val_loss: 0.3213

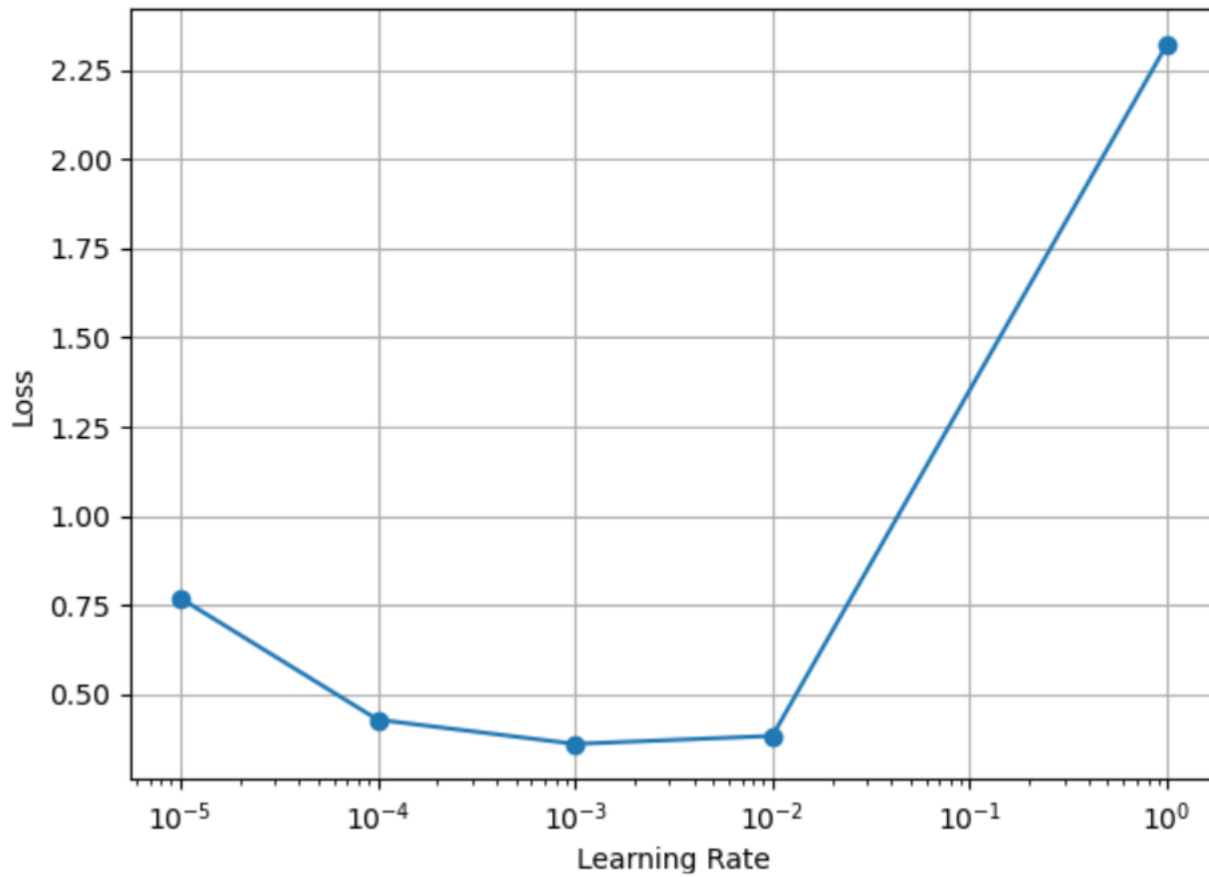
Training Accuracy



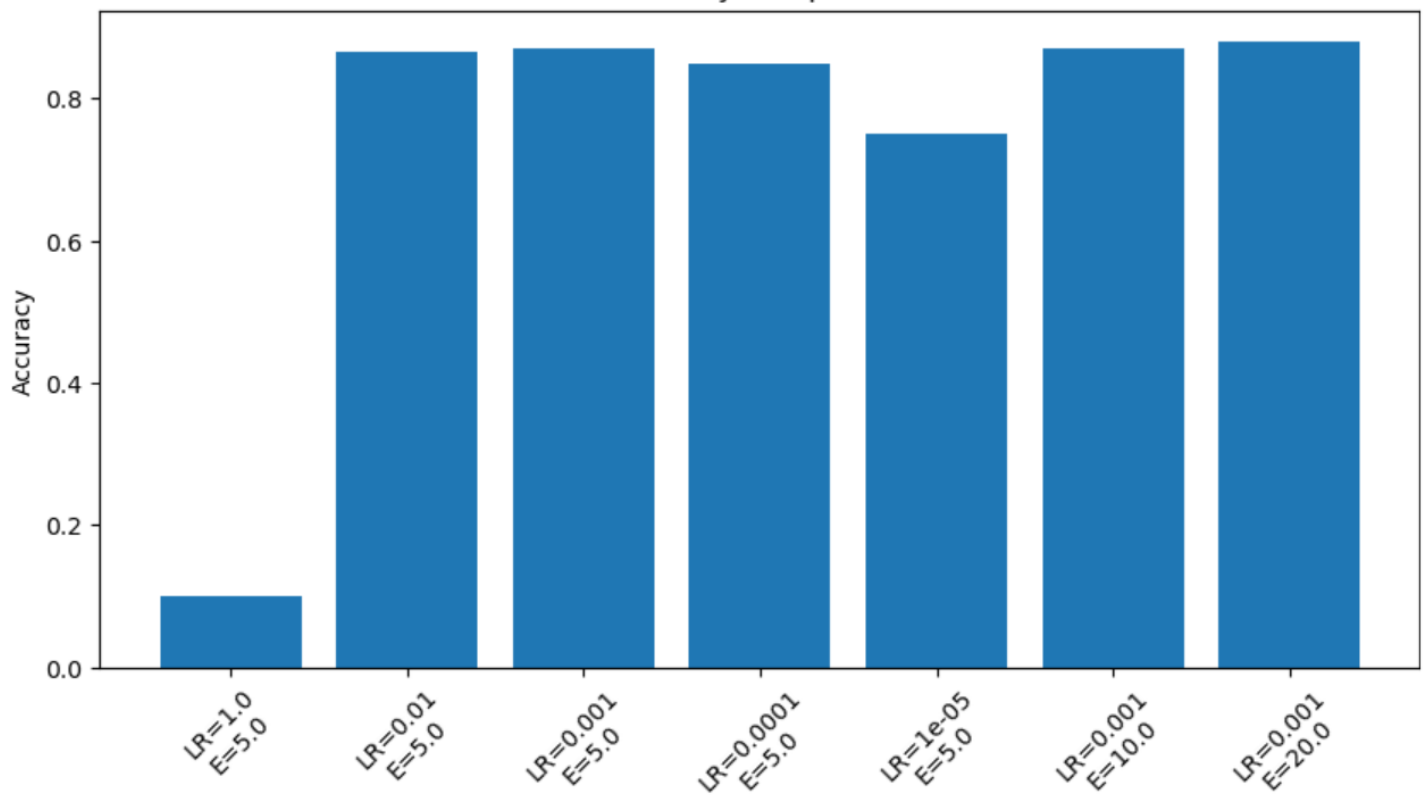
Training Loss



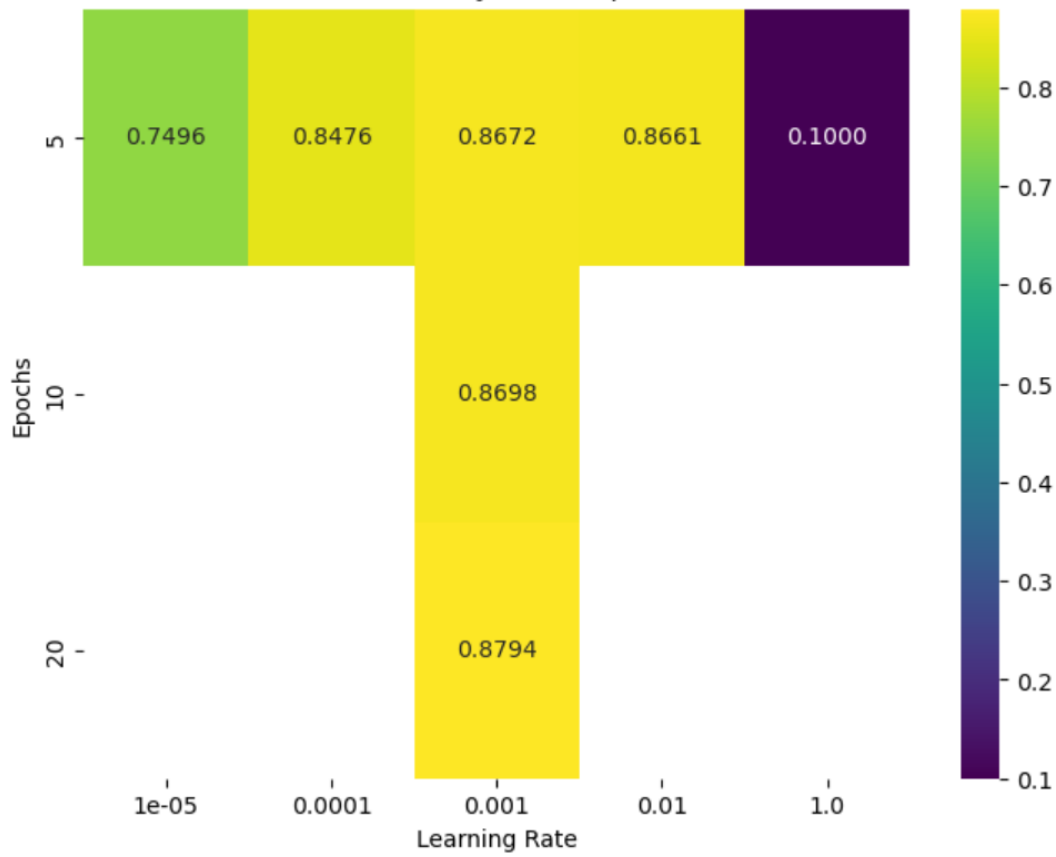
Learning Rate vs Loss



Accuracy Comparison



Accuracy Heatmap



313/313 1s 3ms/step - accuracy: 0.8802 - loss: 0.3706

Final Test Accuracy : 0.8802

Final Test Loss : 0.3706